

Advanced Latency Optimization and Microstructure Architecture for Kalshi Trading Systems

The maturation of prediction markets from rudimentary retail polling mechanisms into institutional-grade financial exchanges has fundamentally altered the microstructure of event-driven trading. Kalshi, operating as a Commodity Futures Trading Commission (CFTC) regulated Designated Contract Market (DCM), provides centralized, off-chain order matching with fiat USD settlement. Unlike decentralized automated market makers (AMMs) that rely on continuous bonding curves and variable blockchain finality, Kalshi functions on a Central Limit Order Book (CLOB) architecture, operating with execution dynamics analogous to traditional financial exchanges like the CME or NASDAQ.

For quantitative trading systems, this structural reality necessitates a paradigm shift. Profitability on Kalshi is no longer dictated solely by superior forecasting accuracy, but by deterministic, ultra-low latency execution infrastructure. When macroeconomic data (such as CPI prints), judicial rulings, or breaking news is released, the order book reprices in single-digit milliseconds. The trading entities that reprice their limit orders first capture the spread, while slower participants suffer from acute adverse selection, filling at unfavorable prices as the market aggressively moves against them.

To operate a trading bot at the absolute physical limits of network transmission, engineers must abandon conventional web development paradigms. A common misconception in API integration is that increasing the frequency of client-side "pings" will yield faster data resolution. This is architecturally incorrect; Kalshi's WebSocket data delivery is an asynchronous, push-based mechanism where heartbeats are strictly server-driven. Optimizing execution speed requires a holistic architectural overhaul spanning physical network topography, protocol selection, operating system kernel tuning, and application-layer memory management.

Demystifying the Kalshi WebSocket Architecture

Achieving terminal velocity in market data ingestion begins with a rigorous understanding of the exchange's real-time streaming protocols. While Kalshi offers a REST API, its utility is strictly limited to bootstrapping initial application state, reconciling settled fills, and executing complex portfolio queries. The REST API utilizes a token-bucket rate-limiting model with independent budgets for Read and Write operations. A Basic tier account is granted 200 Read tokens and 100 Write tokens per second, which scales to 10,000 Read and 8,000 Write tokens per second for Prestige tier participants. Notably, when a rate limit is exceeded, Kalshi returns an HTTP 429 status code with a simple payload and zero Retry-After headers, leaving naive polling systems flying blind and necessitating complex exponential backoff algorithms. Consequently, REST is fundamentally unsuited for high-frequency market data ingestion due to the overhead of establishing Transport Layer Security (TLS) connections and the inherent polling latency. For continuous, real-time data consumption, the WebSocket API serves as the primary interface for most automated participants. The Kalshi WebSocket API establishes a persistent, stateful Transmission Control Protocol (TCP) connection over TLS, streaming continuous updates

without polling overhead. Authentication occurs exclusively during the initial HTTP handshake. The client must generate an RSA-PSS signature using MGF1(SHA256) over the exact string of the timestamp, the HTTP method, and the connection path, passing this signature within three custom headers (KALSHI-ACCESS-KEY, KALSHI-ACCESS-SIGNATURE, KALSHI-ACCESS-TIMESTAMP). Once the socket is successfully upgraded, no further authentication overhead is incurred, allowing for frictionless data streaming.

The operational tempo of the WebSocket connection is entirely dictated by the Kalshi server. The exchange transmits a Ping control frame (0x9) containing the body payload heartbeat exactly every 10 seconds. The client application is strictly required to respond with a Pong frame (0xA) to maintain the connection. Initiating client-side pings at a higher resolution does not accelerate the delivery of market data; it merely consumes bandwidth. If the client fails to respond to the server's heartbeat, or if the connection goes idle without proper lifecycle management, the connection is terminated.

To optimize WebSocket data flow, engineering efforts must be aggressively redirected toward maximizing read throughput on the local socket buffer. During high-volatility events, Kalshi transmits massive bursts of orderbook_delta messages. If a trading bot processes these messages too slowly, the underlying operating system's TCP receive buffer fills up. Kalshi's backend infrastructure detects this backpressure and terminates the connection, emitting a specific error code to indicate that the client's consumption rate is inadequate.

WebSocket Error Code	Designation	Architectural Implication
1	Unable to process message	Malformed JSON or schema incompatibility emitted by the client.
9	Authentication required	Failure in the RSA-PSS handshake or insufficient permissions.
18	Command timeout	The Kalshi server timed out routing a command to a subscription.
25	Subscription buffer overflow	Client read throughput is too slow to handle message bursts.
27	Too many requests	The subscription has exceeded its command rate limit.

Encountering Error Code 25 is the primary indicator of a structurally deficient trading bot. To prevent buffer overflow disconnects, the client's socket reading thread must be entirely decoupled from the strategy execution thread. This is typically achieved by operating the network listener in a lock-free loop that drains the OS socket buffer as rapidly as bytes arrive, passing the raw payloads into an asynchronous ring buffer for downstream processing. Furthermore, the data structures utilized to parse these messages must reflect Kalshi's modern schema. In early 2026, Kalshi deprecated legacy integer cent fields (such as yes_price: 65) in favor of fixed-point string representations. Market data is now delivered with fields such as price_dollars: "0.9600" and delta_fp: "-54.00". Standard floating-point data types (such as Python's float or C++ double) must never be used to parse these values due to the inherent IEEE 754 rounding errors, which will corrupt the local order book state and lead to catastrophic execution failures. High-frequency systems must parse these string decimals directly into custom scaled integers (centi-cents) to enable deterministic, lightning-fast integer arithmetic within the CPU's Arithmetic Logic Unit (ALU).

When subscribing to the `orderbook_delta` channel with a specific `market_ticker`, Kalshi transmits a single `orderbook_snapshot` message to establish the initial state, followed by a continuous, sequenced stream of `orderbook_delta` events. Every snapshot and delta carries a `seq` (sequence) integer. The delta processing algorithm must continuously validate that the incoming `seq` is exactly one integer higher than the previous message. If a sequence number is skipped—indicating a dropped packet somewhere in the network path—the bot must immediately suspend trading, clear the local order book representation, and send an `update_subscription` command with the `get_snapshot` action to trigger a state resynchronization. Operating on a desynchronized order book allows latency arbitrage strategies to execute against phantom liquidity, resulting in guaranteed financial losses.

The FIX Protocol: The Institutional Gateway

While a highly optimized WebSocket implementation is suitable for sophisticated retail algorithms, absolute latency minimization requires transitioning to the Financial Information eXchange (FIX) protocol. Kalshi offers FIXT.1.1 over FIX 5.0 SP2, representing the institutional standard for direct market access and high-frequency trading.

The primary architectural advantage of the Kalshi FIX implementation is its network routing. Standard REST and WebSocket traffic is routed through cloud edge nodes, such as CloudFront, which introduces non-deterministic latency spikes (jitter) depending on global web traffic. A public API call traversing a CDN edge may average 50 milliseconds but can randomly spike to 400 milliseconds. In contrast, FIX sessions connect directly to dedicated order entry and market data hosts, completely bypassing the public cloud edge architecture.

Furthermore, the FIX protocol's tag-value structure (and its associated binary encodings) requires drastically less parsing overhead than the verbose, string-heavy JSON payloads transmitted over WebSockets. This allows for advanced hardware execution techniques, such as parsing Simple Binary Encoding (SBE) directly from network interface cards, reducing message encoding and decoding times from milliseconds to nanoseconds.

Kalshi divides its FIX architecture into highly specialized endpoints, each requiring specific `TargetCompID` values and port configurations. Order entry messages (Tags 35=D, 35=F, 35=G) are routed directly to the matching engine, consuming from the account's Write token bucket.

FIX Endpoint Purpose	Port	TargetCompID	Operational Description
Order Entry (No Retransmission)	8228	KalshiNR	Lowest latency order entry; no message persistence.
Order Entry (Retransmission)	8230	KalshiRT	Order entry with persistence and RFQ creation capabilities.
Drop Copy	8229	KalshiDC	Historical execution report queries for institutional compliance.
Post Trade	8231	KalshiPT	Read-only streaming for settlement reports and position resolution.
Market Data	8233	KalshiMD	Dedicated host for order book snapshots

FIX Endpoint Purpose	Port	TargetCompID	Operational Description
			and incremental updates.

Institutional participants leveraging the KalshiNR (No Retransmission) endpoint achieve the absolute fastest path to the matching engine, as the exchange is not burdened with writing the session state to disk for potential replay requests. FIX sessions are highly stateful and require rigorous sequence number management. Unlike WebSockets, Kalshi does not initiate sequence number resets during maintenance windows; clients must manage their own sequence resets upon reconnecting.

A critical risk management feature available exclusively via FIX is the CancelOrderOnPause behavior. By configuring Tag 21006 on a New Order Single (35=D) message, trading systems can instruct the Kalshi matching engine to automatically cancel resting limit orders if an exchange-wide trading pause occurs. This protects market makers from being executed at stale prices when the market unpauses following a severe volatility halt.

Network Topography and Geographic Proximity

The theoretical limits of execution speed are ultimately constrained by the physical laws of the universe. Light travels through fiber optic cables at approximately 200,000 kilometers per second, meaning every kilometer of physical distance between the trading server and the exchange adds unavoidable latency. Therefore, optimizing the physical network topography is the most critical variable in any high-frequency trading deployment.

Kalshi's core matching engine and backend infrastructure reside within Amazon Web Services (AWS). Specifically, Kalshi operates out of the AWS us-east-2 region located in Ohio. This geographic location is highly strategic, placing it in close proximity to major financial routing hubs in Chicago. Conversely, competitors like Polymarket operate their off-chain matching engines primarily in AWS us-east-1 (Northern Virginia).

To achieve baseline competitiveness on Kalshi, a trading bot cannot be hosted on a residential internet connection. Residential ISPs route traffic through myriad consumer switches, introducing massive jitter and baseline latencies ranging from 50 to 200 milliseconds. Deploying the trading architecture within an enterprise-grade Virtual Private Server (VPS) or a dedicated bare-metal instance geographically proximate to the Ohio exchange reduces latency exponentially. Servers located in specialized Chicago or Ohio data centers achieve round-trip latencies of approximately 1 to 10 milliseconds to Kalshi's infrastructure.

Infrastructure Deployment	Geographic Location	Typical Latency to Engine	Jitter Profile
Residential Internet	Variable	50ms – 200ms	Extremely High
Standard Cloud VPS	New York / Virginia	10ms – 30ms	Moderate
Co-located Trading VPS	Chicago / Ohio	1ms – 10ms	Low
AWS PrivateLink / VPC Peering	AWS us-east-2	< 1ms	Near-Zero

For institutional participants requiring absolute network isolation and minimal latency variance, Kalshi supports AWS PrivateLink for members on the Premier or Prime tiers. By provisioning PrivateLink interface endpoints, the trading system connects to Kalshi via local DNS names within an AWS Virtual Private Cloud (VPC), routing FIX, REST, and WebSocket traffic entirely within the internal AWS backbone.

This architecture completely bypasses the public internet, eliminating external BGP routing inefficiencies, cross-ISP hops, and public CDN bottlenecks. The resulting network-induced jitter is reduced to sub-millisecond standard deviations. Prime tier members can further utilize direct VPC peering for native AWS production connectivity, representing the absolute apex of topographical optimization, practically placing the trading bot inside the same logical data center as the exchange's matching engine.

Taming the Linux Kernel: Operating System Tuning

Even with optimal AWS co-location and FIX protocol integration, the default Linux operating system network stack introduces significant latency. A standard Linux kernel is optimized for high throughput, background process fairness, and energy efficiency, not for the microsecond determinism required by high-frequency financial trading systems.

When a network packet arrives at a standard Linux server, the Network Interface Card (NIC) generates a hardware interrupt. The CPU must halt its current execution, perform a costly context switch, and execute an interrupt handler to move the packet into a socket buffer (`sk_buff`), before finally waking the user-space application to read the data. This process takes tens of microseconds and introduces severe jitter. To ensure that market data is ingested and orders are dispatched instantaneously, the entire operating system must be aggressively tuned.

TCP/IP Socket Optimization via Sysctl

The behavior of the Linux network stack is governed by parameters configured via `sysctl.conf`. The most critical adjustment is disabling Nagle's algorithm. By default, TCP employs Nagle's algorithm, which buffers small packets to group them into larger payloads, conserving bandwidth at the expense of latency. In trading, holding a 200-byte order submission for even one millisecond to wait for additional data is fatal. Enabling the `TCP_NODELAY` socket option forces the kernel to dispatch packets immediately onto the wire.

To eliminate the overhead of hardware interrupts during market data ingestion, modern Linux kernels support Socket Busy Polling (`SO_BUSY_POLL`). This socket option allows the user-space application to directly poll the network device's receive queue. By keeping the CPU actively spinning on the socket, the system eliminates interrupt handling and context switch overhead. Configuring the global parameter `sysctl net.core.busy_poll=50` (measured in microseconds) drastically reduces jitter on rapid WebSocket or FIX data ingestion. Furthermore, the OS TCP read and write buffers must be expanded to prevent dropping packets during extreme market volatility. Setting the parameters `net.core.rmem_max`, `net.core.wmem_max`, `net.ipv4.tcp_rmem`, and `net.ipv4.tcp_wmem` to 64MB allows the kernel to absorb sudden bursts of order book updates (such as those following a major news release) without triggering Kalshi's Error 25 buffer overflow disconnects. Accelerating connection establishment is also vital for rapid recovery after a disconnect; enabling `net.ipv4.tcp_tw_reuse` allows the rapid recycling of sockets in the `TIME_WAIT` state, while `net.ipv4.tcp_fastopen=3` reduces the three-way handshake overhead for previously connected hosts.

Processor Scheduling and CPU Isolation

A CPU core that switches between the trading application, background OS tasks, and network interrupts creates unpredictable latency spikes. System administrators must partition the

hardware to dedicate specific physical cores exclusively to the trading bot, isolating them from the general operating system environment.

The foundational step is utilizing the kernel boot parameter `isolcpus`, which removes specific CPU cores from the Linux scheduler's default load balancing algorithms. Using utilities like `taskset` or native thread affinity APIs within the application, the critical order-matching and network-reading threads are explicitly pinned to these isolated cores.

To further protect these cores, the `irqbalance` daemon, which dynamically moves hardware interrupts across CPU cores to balance power consumption, must be completely disabled. System interrupts (such as those from storage drives or management network interfaces) must be manually routed to the non-isolated cores via `/proc/irq/*/smp_affinity`, leaving the trading cores completely uninterrupted.

Linux traditionally relies on a timer interrupt (tick) to manage process scheduling and timekeeping, defaulting to 250Hz or 1000Hz depending on the preemption model. This means the CPU is interrupted hundreds of times a second simply to check the time. By compiling the kernel with `CONFIG_NO_HZ_FULL` and passing `nohz_full` alongside `rcu_nocbs` in the boot parameters, the kernel stops sending timer interrupts to the isolated cores entirely, provided only a single runnable task is active on that core.

At the hardware BIOS level, Hyper-Threading (Simultaneous Multithreading or SMT) must be disabled. SMT forces two logical threads to share the physical core's L1/L2 cache and execution units, introducing microscopic contention. Disabling SMT guarantees the trading thread has exclusive, uncontested access to the physical core's hardware resources.

Linux Tuning Strategy	Configuration Method	Architectural Purpose
TCP_NODELAY	Application Socket	Disables Nagle's algorithm; forces immediate packet dispatch.
SO_BUSY_POLL	sysctl / Socket API	Replaces interrupt-driven I/O with CPU busy-polling.
TCP Buffer Scaling	sysctl.conf	Absorbs market data bursts; prevents Buffer Overflow errors.
isolcpus / taskset	Boot param / OS command	Dedicates physical CPU cores entirely to the trading application.
Tickless Kernel (nohz_full)	Kernel Compilation / Boot	Stops periodic OS timer interrupts on isolated cores.
Disable Hyper-Threading	BIOS Configuration	Prevents cache contention between logical threads on one core.

Extreme Latency Reduction: Kernel Bypass and FPGA Acceleration

Even a perfectly tuned Linux kernel imposes fundamental speed limits due to the inherent architecture of the POSIX socket API. Software operating on a standard CPU relying on OS network stacks yields 5 to 50 microseconds of processing latency. For the elite echelon of market makers, this is unacceptably slow.

To achieve sub-microsecond latency, the Linux kernel network stack must be bypassed entirely.

Kernel bypass frameworks map the memory of the Network Interface Card (NIC) directly into the user-space application's memory footprint.

The most prominent technologies in this domain are Solarflare OpenOnload, the Data Plane Development Kit (DPDK), and eBPF/AF_XDP. OpenOnload is particularly highly regarded in financial engineering because it transparently intercepts standard POSIX socket API calls (like `recv()` and `send()`). This means a trading bot's existing WebSocket or FIX client can be accelerated to hardware speeds without requiring a complete rewrite of the underlying application logic.

By utilizing DPDK or OpenOnload with poll-mode drivers, incoming network packets travel from the wire directly into the trading application's L1/L2 CPU cache. This architecture completely bypasses the kernel interrupt handlers, the IP routing layer, the TCP state machine, and the `sk_buff` memory allocations, pushing internal tick-to-trade latencies below 1 microsecond.

For operations requiring speeds measured in nanoseconds, software execution must be abandoned entirely in favor of Field Programmable Gate Arrays (FPGAs). An FPGA is custom-programmed hardware, utilizing languages like VHDL or Verilog, to process market data feeds and execute trading logic directly on the silicon. FPGA designs utilize lock-free hardware pipelines and cache line optimization, achieving execution speeds up to 50 times faster than highly optimized C++ code running on a CPU.

Technology Stack	Typical Packet Latency	Flexibility	Development Cost / Complexity
Standard Linux Sockets	50 – 100 μ s	Very High	Low (Standard APIs)
Tuned Linux (SO_BUSY_POLL)	10 – 50 μ s	High	Medium (Requires OS Admin)
Kernel Bypass (OpenOnload / DPDK)	< 5 μ s	Moderate	High (Requires specific NICs)
FPGA (Hardware Pipeline)	< 1 μ s	Low	Extremely High (VHDL/Verilog)

If an organization relies on software rather than FPGAs, the application layer must enforce strict memory determinism. Languages with non-deterministic garbage collection (GC), such as Python, are unsuitable for the critical path of an HFT system. A Python garbage collection pause can stall the application for tens of milliseconds, completely nullifying all network and OS tuning. High-performance infrastructure is predominantly written in C++ or Rust, leveraging zero-cost abstractions, minimal allocations in hot paths, and highly optimized BTree structures for order book sorting. If Java is mandated by organizational requirements, it must be authored using aggressive "zero-allocation" patterns, pre-allocating all memory buffers at startup to prevent GC events during active trading.

Thread synchronization within the application must also avoid kernel intervention. Traditional lock-based synchronization (e.g., standard mutexes) forces threads to sleep when waiting for resources, requiring a kernel context switch to wake them up. High-speed trading bots utilize lock-free data structures, most notably the LMAX Disruptor pattern or single-producer, single-consumer (SPSC) ring buffers. The network thread writes market data updates to the ring buffer using atomic memory barriers, and the strategy thread reads from it using busy-spin waits. This architecture keeps both threads infinitely looping on their respective pinned CPU cores, ensuring that data is transferred between threads entirely within the CPU's ultra-fast L3 cache, minimizing slower RAM access times.

Microstructure Implications, Latency Arbitrage, and

Manipulation

Constructing an execution engine capable of blistering speed yields profound, asymmetric advantages in the microstructural environment of a prediction market. The continuous injection of institutional liquidity and automated quoting into Kalshi fundamentally alters market dynamics, directly penalizing slower participants.

The Mechanics of Adverse Selection and Latency Arbitrage

In prediction markets, contracts represent probabilities of real-world events. When an exogenous shock occurs, the true probability of the event changes instantaneously. However, the limit orders resting on the Kalshi order book will not update until the algorithms managing them receive the news, parse it, and dispatch cancel commands over the network.

A highly optimized bot utilizing kernel bypass and AWS co-location will receive the news feed, calculate the new probability, and route aggressive Taker orders via the KalshiNR FIX endpoint to the matching engine before slower Market Makers can cancel their resting orders. The slower participants suffer from adverse selection, inadvertently buying or selling contracts at outdated prices. This dynamic highlights why speed is not just about executing a strategy, but about defensive survival; a slow market maker on Kalshi will be systematically exploited by faster participants. Market makers counter this by widening spreads during periods of anticipated volatility or by utilizing the CancelOrderOnPause FIX feature to withdraw liquidity the millisecond a market halt is triggered.

Cross-Venue Information Discovery

The deployment of low-latency infrastructure opens the door to cross-platform statistical arbitrage, specifically between Kalshi (settled in fiat, regulated in the U.S.) and decentralized counterparts like Polymarket (settled in crypto, operating globally).

While Polymarket operates conceptually on the Polygon blockchain, its order book matching is performed off-chain on centralized servers (often located in AWS us-east-1), closely mirroring Kalshi's limit order book structure. Because prediction markets are highly shock-driven, pricing discrepancies between the two venues emerge frequently. An architecture optimized for Kalshi can simultaneously ingest Polymarket's WebSocket feed. When a large block trade consumes liquidity and moves the price on Polymarket, the HFT bot can instantly recognize the pricing disparity and execute a corresponding hedge on Kalshi. Spreads that appear exploitable at a 3-cent difference will vanish in less than a second as automated systems compete to capture the arbitrage; hence, the sub-millisecond execution capabilities provided by FIX and OpenOnload are absolute requirements for capitalizing on these fleeting inefficiencies.

Manipulation and Settlement Vulnerabilities

An essential third-order effect of enhanced execution speed in prediction markets is the capacity to interact with "asset-price contracts." These are binary contracts that resolve based on the price of a financial asset (e.g., the price of Bitcoin at the end of a specific 15-minute window). Academic models and empirical evidence indicate that these short-horizon prediction contracts are highly susceptible to cash-settlement manipulation. A market participant can purchase a prediction contract, and in the final milliseconds before settlement, use an ultra-fast algorithmic

execution system to dump a massive order on the underlying spot market (e.g., Binance). This sudden burst of spot volume pushes the oracle price over the strike threshold, flipping the resolution of the binary contract in their favor, before the spot price naturally reverts seconds later.

An algorithmic trading bot operating on Kalshi must be programmed to anticipate these micro-structural anomalies. As the settlement window approaches, liquidity on asset-price contracts tends to evaporate or become highly toxic, as informed manipulators prepare to push the spot price. A sophisticated trading system will dynamically widen its quoted spreads or pull its resting orders entirely in the final seconds before resolution, utilizing its extreme latency advantage to exit the market before manipulative spot-price pushes trigger adverse fills against its positions.

In conclusion, ensuring that a trading bot perceives and analyzes Kalshi market data at absolute maximum velocity is a multi-disciplinary engineering challenge. It requires abandoning polling APIs in favor of institutional FIX connections, bypassing cloud CDNs for AWS PrivateLink co-location, stripping the Linux kernel of its interrupt-driven inefficiencies via busy-polling and kernel bypass, and architecting the application layer with lock-free, zero-allocation memory paradigms. By integrating these methodologies, a trading entity transitions from a passive participant vulnerable to adverse selection into an apex system capable of executing latency arbitrage, providing resilient market-making liquidity, and surviving the brutal microstructural realities of modern prediction markets.

Works cited

1. Kalshi API Guide: Python SDK Setup, RSA Auth & Demo Sandbox, <https://agentbets.ai/guides/kalshi-api-guide/>
2. Best VPS for Kalshi Trading Bots (2026) - TradoxVPS, <https://tradoxvps.com/best-vps-for-kalshi-trading-bots/>
3. OpenMarket: A Synchronized Polymarket–Binance Dataset for High, <https://arxiv.org/html/2607.26245v1>
4. A SWE-bench-Style Framework for Backtesting Trading Agents on, <https://arxiv.org/pdf/2602.00133>
5. What Happens When Institutional Liquidity Enters Prediction ... - arXiv, <https://arxiv.org/html/2604.10005v2>
6. Kalshi API: The Complete Developer Guide (2026) - Parlay, <https://www.parlay.run/kalshi-api>
7. Kalshi WebSocket API: Rate Limits, Orderbook and Examples, <https://blog.predictify.com/kalshi-websocket-api>
8. Kalshi API Guide: Market Data, Auth & Rate Limits, <https://www.botforkalshi.com/blog/kalshi-api-tutorial>
9. Polymarket API for Trading Bots: Rate Limits & WebSockets, <https://tradoxvps.com/polymarket-api-for-trading-bots/>
10. Quick Start: WebSockets - Kalshi's API Documentation, https://docs.kalshi.com/getting_started/quick_start_websockets
11. Design and Implementation of a Low-Latency High-Frequency, <https://medium.com/@gwrx2005/design-and-implementation-of-a-low-latency-high-frequency-trading-system-for-cryptocurrency-markets-a1034fe33d97>
12. Orderbook Updates - Kalshi's API Documentation, <https://docs.kalshi.com/websockets/orderbook-updates>
13. kalshi_trading - Rust - Docs.rs, <https://docs.rs/kalshi-trading>
14. API Changelog - Kalshi's API Documentation, <https://docs.kalshi.com/changelog>
15. Kalshi API: The Complete Developer's Guide - DEV Community, <https://dev.to/zuplo/kalshi-api-the-complete-developers-guide-1fo4>
16. Connectivity - Kalshi's API Documentation, <https://docs.kalshi.com/fix/connectivity>
17. How fast are FIX APIs in comparison to WebSocket APIs? I cannot, https://www.reddit.com/r/algotrading/comments/8so6nu/how_fast_are_fix_apis_in_comparison_to_websocket/
18. Running Kalshi Bots on a VPS: Why Low Latency Matters - QuantVPS, <https://www.quantvps.com/blog/running-kalshi-bots-on-a-vps>
19. Best polymarket VPS where

priority is low latency - Reddit,
https://www.reddit.com/r/algotrading/comments/1r68xm3/best_polymarket_vps_where_priority_is_low_latency/ 20. Best VPS for Trading: Top 19 Providers for Low Latency & Reliability,
<https://www.quantvps.com/blog/best-vps-for-trading> 21. Best VPS for Polymarket bot? (Low latency + Cloudflare-friendly),
https://www.reddit.com/r/VPS/comments/1pw3qvn/best_vps_for_polymarket_bot_low_latency/ 22. API Environments and Endpoints - Kalshi's API Documentation,
https://docs.kalshi.com/getting_started/api_environments 23. Guide to Low Latency Linux Kernel Configurations - Baeldung, <https://www.baeldung.com/linux/kernel-low-latency-settings> 24. Expert Guide to Sysctl Performance Tuning in Linux,
<https://linuxgd.medium.com/expert-guide-to-sysctl-performance-tuning-in-linux-658366c6fdeb> 25. Kernel Bypass Techniques in Linux for High-Frequency Trading,
<https://lambdafunc.medium.com/kernel-bypass-techniques-in-linux-for-high-frequency-trading-a-deep-dive-de347ccd5407> 26. Optimizing TCP for high WAN throughput while preserving low latency, <https://blog.cloudflare.com/optimizing-tcp-for-high-throughput-and-low-latency/> 27. Chapter 32. Improving network latency using TCP_NODELAY,
https://docs.redhat.com/en/documentation/red_hat_enterprise_linux_for_real_time/9/html/optimizing_rhel_9_for_real_time_for_low_latency_operation/assembly_improving-network-latency-using-tcp_nodelay_optimizing-rhel9-for-real-time-for-low-latency-operation 28. How to use SO_BUSY_POLL socket option,
https://b2bits.atlassian.net/wiki/spaces/B2BITS/pages/6067472/How+to+use+SO_BUSY_POLL+socket+option 29. Linux Performance Optimization - Supercharged Computing,
<https://superchargedcomputing.com/2018/03/02/linux-optimization/> 30. Linux Tuning - Fasterdata, <https://fasterdata.es.net/host-tuning/linux/> 31. Low Latency Tuning Guide | Erik Rigtorp, <https://rigtorp.se/low-latency-guide/> 32. Using Immutable Ledgers to Secure Financial Data on Linux Platforms,
<https://www.tmcnet.com/topics/articles/2026/04/10/463453-hardening-trading-infrastructure-using-immutable-ledgers-secure-financial.htm> 33. Ultra-Low-Latency Trading Bot Development with AI, FPGA, and Co,
<https://truetech.dev/ai-development/services/industry-solutions/ai-hft-trading-bot.html> 34. Exchange Infrastructure Engineering | AI Agent Skill,
<https://www.thepromptindex.com/skill/1463-exchange-infrastructure> 35. aws-samples/trading-latency-benchmark - GitHub,
<https://github.com/aws-samples/trading-latency-benchmark> 36. Running Ember with Xilinx Onload TCP Bypass,
https://ember.deltixlab.com/docs/performance/running_ember_with_onload/ 37. Rust vs C++, which is best for building a high frequency trading,
<https://users.rust-lang.org/t/rust-vs-c-which-is-best-for-building-a-high-frequency-trading-software-from-scratch/137436> 38. Principal Trading Engineer (Java, Algo, SOR) - Fidelity Careers,
<https://jobs.fidelity.com/en/jobs/2115908/principal-trading-engineer-java-algo-sor/> 39. Settlement Manipulation in Prediction Markets - arXiv, <https://arxiv.org/html/2606.31675v1>